Deep Reinforcement Learning

Monte Carlo Methods

Prof. Dr. Sebastian Peitz

Chair of Safe Autonomous Systems, TU Dortmund

Summer term 2026

Monte Carlo simulation

What does *Monte Carlo sampling* mean?

Monte Carlo Port (Source)

- We're trying to estimate the expected value of function of a random variable $x = X(\omega)$:

$$\mathbb{E}\left[f(X)\right] = \sum_{\omega \in \Omega} p(x) f(x) \qquad / \qquad \mathbb{E}\left[f(X)\right] = \int_{\omega \in \Omega} p(x) f(x) \, \mathrm{d}x.$$

- Not knowing f or the random variable X, we need to rely on *experience*.
- Monte Carlo sampling: Having no prior knowledge, we can estimate the expected value from repeated experiments.
- This lecture: **Let's use this for (episodic, i.e., $T < \infty$) RL when we don't know the MDP!**

Example: Estimating the value for π

- The area of a quarter circle with radius $r = 1$:

$$A = \frac{1}{4}\pi r^2 = \frac{\pi}{4}.$$

- Randomly and uniformly place points in the unit square:

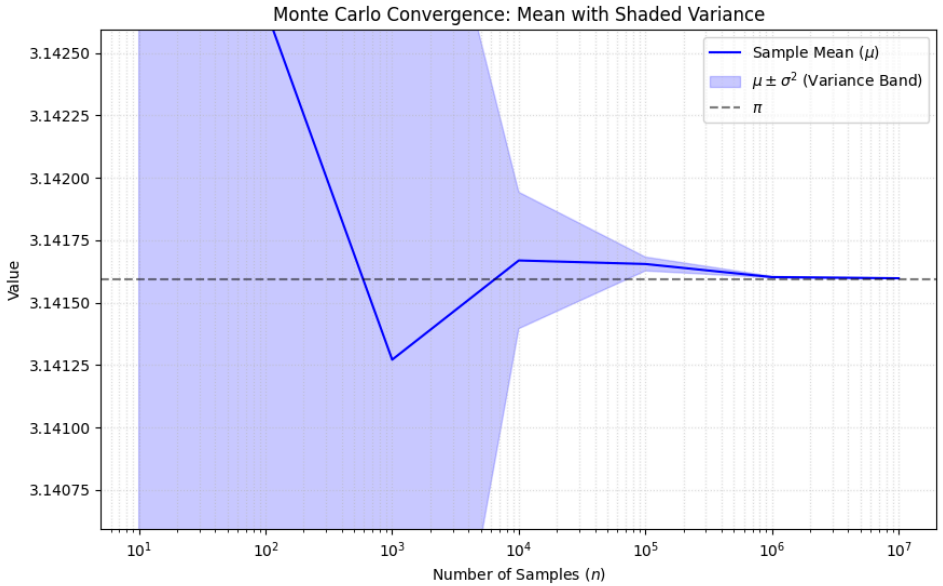
$$(x, y) \sim U([0, 1]^2).$$

- Probability of landing **inside** the quarter circle is the ratio of the areas:

$$p(x^2 + y^2 \leq 1) = \frac{A}{1^2} = \frac{\pi}{4}.$$

- Monte Carlo estimate:

$$\pi = 4 \cdot \mathbb{E}\left[p(x^2 + y^2 \leq 1)\right].$$



Monte Carlo prediction

Monte Carlo prediction

We begin by learning V^π for a given policy π.

- Recall that $V^\pi(s)$ is the expected return (expected cumulative future discounted reward) starting from that state:

$$V(s) = \mathbb{E}\left[g_t | s_t = s\right] = \mathbb{E}\left[\sum_{k=0}^{T} \gamma^k r_{t+k} \middle| s_t = s\right].$$

- Simple way to estimate from experience: average the returns observed after visits to a state s.
- As more returns are observed, the average should converge to the expected value.
- Two different approaches:
  - First-visit MC: approximate the return only from the first visit to a state s.
  - Every-visit MC: calculate an update for the $V(s)$ estimate for every visit to a state s.

Algorithm: First-visit MC prediction for estimating $V \approx V^\pi$.
initialize

- $V(s)$ arbitrarily for $s \in \mathcal{S}$
- $\ell(s)$: an empty list of returns for all $s \in \mathcal{S}$

for $j = 1, 2, \dots, J$ episodes:
  $g = 0$
  Generate a sequence following π:

$$((s_0, a_0, r_0), (s_1, a_1, r_1), \dots, (s_{T_j-1}, a_{T_j-1}, r_{T_j-1}))$$

  for $t \in \{T_j - 1, T_j - 2, T_j - 3, \dots, 0\}$:
    $g = \gamma g + r_t$
    if $s_t \notin \{s_0, \dots, s_{t-1}\}$: (*that's the first-visit condition*)
      Append g to $\ell(s_t)$
      $V(s_t) =$ **average**$(\ell(s_t))$

Convergence? Yes, for every state $s \in \mathcal{S}$ that is visited infinitely often ($\rightarrow$ exploration!)

Advantages of MC methods

1. We can learn from experience, even if we do not know the model

- actual experience,
- simulated experience, if we have a model (but do not necessarily know the transition probability $p\left(s'|s,a\right)$).

2. The estimates for each state are independent of each other
   $\Rightarrow$ we are not bootstrapping, i.e., building estimates based on other estimates.

3. As a consequence, we can estimate parts of the value function that are of particular interest.

Example: Gridworld

TBD

Monte Carlo estimation of action values

Monte Carlo estimation of action values

Is a model available (i.e., the MDP $(\mathcal{S}, \mathcal{A}, p, r, \gamma)$ is known)?

- Yes $\Rightarrow$ state values plus one-step predictions deliver an optimal policy.
- No $\Rightarrow$ action values are very useful to directly obtain optimal policies.

Estimating $Q^\pi(s, a)$ using MC:

- Analog to the MC prediction algorithm for $V^\pi(s)$.
- Small variation: A visit refers to a state-action pair (s, a).
- First-visit and every-visit variants exist

Possible challenges?

- Certain state-action pairs (s, a) may never be visited.
- Missing degree of exploration (in particular for deterministic π).
- Workaraound: **exploring starts** $\Rightarrow$ start episodes in random state-action pairs (s, a).

Monte Carlo control

Monte Carlo control (1)

We learned about the Generalized policy iteration (GPI) in the last lecture.

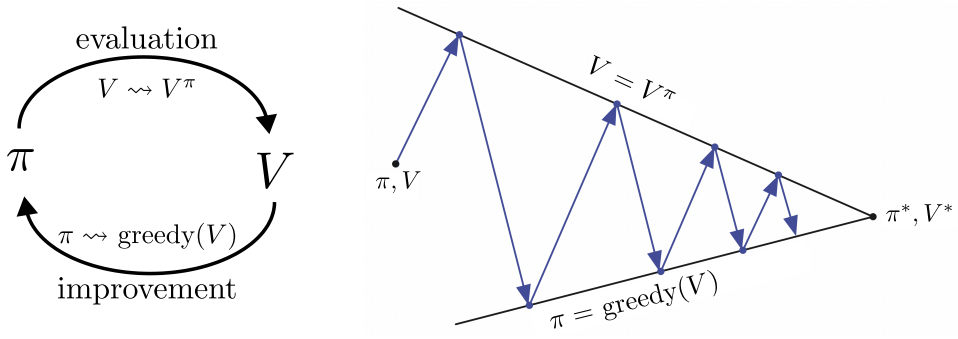
Let's use the same procedure here:

$$\pi_0 \xrightarrow{E} Q^{\pi_0} \xrightarrow{I} \pi_1 \xrightarrow{E} Q^{\pi_1} \ldots \xrightarrow{I} \pi^* \xrightarrow{E} Q^* = Q^{\pi^*}.$$

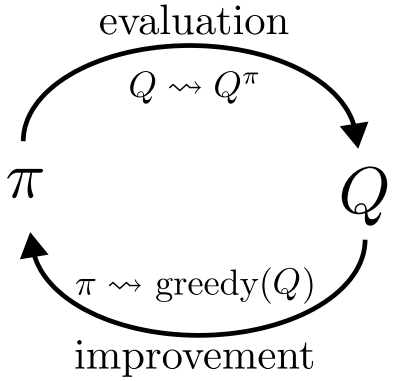
Policy improvement: Make the policy greedy w.r.t. the current value function:

Since we have the *action-value function* Q, we don't need a model to construct the policy:

$$\pi(s) = \arg\max_{a \in \mathcal{A}} Q(s, a).$$



GPI (Sutton and Barto 1998)



GPI for MC control (Sutton and Barto 1998)

Monte Carlo control (2)

Even if we're operating in an unknown MDP, the policy improvement theorem remains valid for MC-based control due to the underlying MDP structure:

Policy improvement theorem for MC-based control

$$
\begin{aligned}
Q^{\pi_k}(s, \pi_{k+1}(s)) &= Q^{\pi_k}\left(s, \arg\max_{a \in \mathcal{A}} Q^{\pi_k}(s, a)\right) \\
&= \max_{a \in \mathcal{A}} Q^{\pi_k}(s, a) \\
&\geq Q^{\pi_k}(s, \pi_k(s)) \\
&\geq V^{\pi_k}(s).
\end{aligned}
$$

- **Policy improvement**: Construct the greedy policy $\pi_{k+1}(s)$ w.r.t. the current approximation Q^{π_k}
- **Assumptions required**:
  - The episodes have epxloring starts. *(We will consider this later)*
  - We are training on an infinite number of episodes.

Monte Carlo control (3)

Algorithm: Monte Carlo ES (Exploring Starts) for estimating $\pi \approx \pi^*$.
initialize

- $\pi(s)$ arbitrarily for $s \in \mathcal{S}$
- $Q(s, a)$ arbitrarily for $s \in \mathcal{S}$, $a \in \mathcal{A}$
- $\ell(s, a)$: an empty list of returns for all $s \in \mathcal{S}$, $a \in \mathcal{A}$

for $j = 1, 2, \dots, J$ episodes:
  $g = 0$
  Choose $s_0 \in \mathcal{S}$ and $a_0 \in \mathcal{A}$ randomly such that all pairs have probability > 0
  Generate a sequence starting at (s_0, a_0) and following π:

$$((s_0, a_0, r_0), (s_1, a_1, r_1), \dots, (s_{T_j-1}, a_{T_j-1}, r_{T_j-1}))$$

  for $t \in \{T_j - 1, T_j - 2, T_j - 3, \dots, 0\}$:
    $g = \gamma g + r_t$
    if $(s_t, a_t) \notin \{(s_0, a_0), \dots, (s_{t-1}, a_{t-1})\}$:
      Append g to $\ell(s_t)$

- It is clear that this algorithm cannot converge to a suboptimal policy
- Stability is only achieved when both π and Q are optimal

$$Q(s_t, a_t) = \text{average}(\ell(s_t, a_t))$$
$$\pi(s_t) = \arg\max_{a \in \mathcal{A}} Q(s_t, a)$$

Example: Gridworld

TBD

On-policy and off-policy learning

On-policy versus off-policy (1)

On-Policy: "Learning by Doing"

- The agent only learns from what it's currently doing.
- It evaluates and improves the exact same policy it is using to make decisions.
- For example,
  - if it is being cautious, it learns how to improve cautious behavior.
  - if it tries something risky and fails, it learns from that failure but can't easily "imagine" what would have happened if it had taken a different action earlier.

Off-Policy: "Learning by Observing"

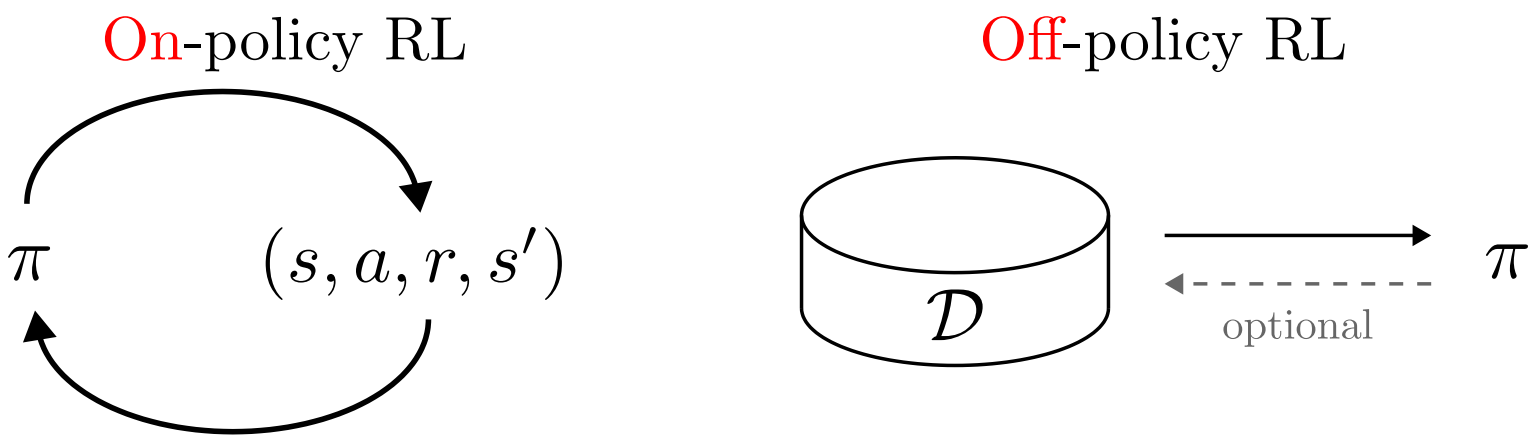
- is more like a student watching an expert or looking at a variety of different actors once.
- It can follow one path (the "behavior policy") while learning the best possible path (the "target policy").
- It separates "how I am acting" from "what I am learning."
- Benefit: It can learn from old data, from human demonstrations, or by watching other agents.

Question: Which class does the *Monte Carlo ES* algorithm belong to?

Answer: It is on-policy! Each time we update our policy (the counter j for the episodes), we collect the returns anew, following the current policy π.

We will cover the topic on-policy versus off-policy in more detail later in this course.

On-policy versus off-policy (2)



Feature	On-policy	Off-policy
Learning Source	Learns from its current actions only.	Can learn from any data (past, random, or expert).
Exploration	Often more stable but can get stuck in “safe” habits.	More aggressive at finding the absolute best strategy.
Efficiency	Lower. Throws away data once the policy changes.	Higher. Can “re-use” old experiences (Experience Replay).

Key differences at a glance:

ϵ-greedy as an on-policy alternative (1)

Motivating question: How do we get rid of the restrictive (and often not achievable) requirement of *exploring starts (ES)*?

- We need to make sure that we visit all state-action pairs, irrespective of where we start!
- Exploration requirement:

$$\pi(a|s) > 0 \quad \forall s \in \mathcal{S}, a \in \mathcal{A}.$$

- Policies of this type are called **soft**.
- The level of exploration should be tunable during the learning process.

Where have we seen something like this before? $\Rightarrow$ In the multi-armed bandits lecture!

ϵ-greedy on-policy learning

- With probability ϵ, the agent's choice (i.e., the policy output) is overwritten by a random action.
- Probability of all non-greedy actions: $\frac{\epsilon}{|\mathcal{A}|}$.
- Probability of the greedy action: $1 - \epsilon + \frac{\epsilon}{|\mathcal{A}|}$.

ϵ-greedy as an on-policy alternative (2)

Algorithm: On-policy first-visit MC control (for ϵ-soft policies).
parameter: small $\epsilon > 0$

initialize

- Arbitrary ϵ-soft policy π
- $Q(s, a)$ arbitrarily for $s \in \mathcal{S}$, $a \in \mathcal{A}$
- $\ell(s, a)$: an empty list of returns for all $s \in \mathcal{S}$, $a \in \mathcal{A}$

Off-policy Monte Carlo control

Off-policy Monte Carlo control

Summary / what you have learned

Summary / what you have learned

References

Sutton, R. S., and A. G. Barto. 1998. *Reinforcement Learning: An Introduction*. MIT Press.